

Technical Portfolio Overview: Self-Correcting SEC Filing Analyst

Executive Summary:

Most enterprise Retrieval-Augmented Generation (RAG) applications fail in quantitative domains because traditional vector similarity retrieves semantically related text that frequently lacks the exact numerical data needed. This causes standard LLMs to hallucinate or interpolate missing numbers.

To solve this, I engineered a deterministic, self-correcting RAG workflow using LangGraph, Gemini 3.6 Flash, and SEC EDGAR real-time filings. The system programmatically verifies numerical grounding, rewrites queries dynamically upon retrieval failure, and guarantees zero ungrounded financial claims.

Technical Architecture & Node Pipeline



Technical Portfolio Overview: Self-Correcting SEC Filing Analyst

Core Architecture Components

- **Upstream Guardrail (validate_and_suggest_ticker):**

Pulls the live SEC company directory (~10,000+ active entities) directly into local cache. Uses hybrid fuzzy token matching combined with LLM semantic fallback to correct misspelled tickers or map raw company names to valid symbols before any processing starts.

- **Retrieval Node (retrieve_node):**

Downloads Form 10-K directly from SEC EDGAR via programmatic user-agent headers. Strips non-content styling, segments document boundaries, and runs an in-memory **BM25Okapi** lexical search. This preserves exact numerical tokens and accounting terminology (\$383,285M, Net Sales) far more reliably than dense semantic embeddings.

- **Document Grader (grade_documents_node):**

Uses Pydantic structured output (GradeResult) to determine whether extracted text contains actual financial metrics or irrelevant risk disclaimers. Acts as a **conditional edge** router.

- **Query Rewriter (rewrite_query_node):**

If the Grader votes no, the query is rewritten into standardized GAAP line-item taxonomy and re-routed to the Retrieval Node. A loop limit (loop_count < 3) serves as a strict circuit breaker against infinite loops.

- **Generation Node (generate_node):**

Synthesizes raw filing text into a structured, executive-ready dashboard: High-level summary, comparison Markdown table, and key strategic takeaways.

- **Hallucination Checker (check_hallucination_node):**

A post-generation cross-check that verifies every single figure in the final report against the source context chunks. If an ungrounded figure is found, it triggers deterministic re-generation.

Key Competencies Demonstrated

- **Orchestration:** Advanced state management with langgraph.graph.StateGraph using cyclical graphs and conditional edges.
- **Cost & Latency Control:** Low-overhead lexical search with BM25, bounded retry loops, and rapid execution using Google Gemini 3.6 Flash.
- **Data Engineering:** Automated SEC EDGAR ingestion, HTML parsing with BeautifulSoup, and structured schema enforcement via Pydantic.